

Trabalho Prático 2 - Sistema de Despacho de Transporte por Aplicativo

Igor Henrique Martins de Almeida

2023028536

Departamento de Ciência da Computação - Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte - MG - Brasil

igorhma@ufmg.br

1. Introdução

Esta documentação lida com o problema do despacho de veículos para transporte por aplicativo, simulando a operação de uma frota que oferece a modalidade de corridas compartilhadas. O cenário proposto envolve a alocação eficiente de demandas de passageiros a veículos, utilizando critérios específicos de capacidade, intervalo temporal e proximidade geográfica para agrupar solicitações e otimizar o uso dos recursos disponíveis. O objetivo desta documentação é apresentar a solução desenvolvida para o problema dado, detalhando a implementação de um Simulador de Eventos Discretos (SED) em C++ capaz de processar a entrada de demandas, formar grupos de passageiros viáveis e simular a execução das corridas ao longo do tempo.

Para resolver o problema, foi seguida uma abordagem modular com o código organizado em diretórios específicos para cabeçalhos, fontes e executáveis. A solução utiliza Tipos Abstratos de Dados (TADs) para representar as entidades do sistema, como Demandas, Corridas e o Escalonador, implementados como classes que gerenciam suas próprias propriedades e estados. Foram empregadas estruturas de dados dinâmicas, incluindo um vetor redimensionável e uma fila de prioridade baseada em *Min-Heap*, para gerenciar eficientemente o fluxo de eventos e o armazenamento de dados durante a simulação. Além da descrição técnica, o documento aborda a análise de complexidade dos algoritmos de agrupamento e escalonamento, as estratégias de robustez para garantir a estabilidade do programa e a análise experimental sobre o impacto dos parâmetros de configuração na eficiência do sistema.

A documentação está organizada da seguinte forma: a Seção 2 descreve em detalhes a implementação, as estruturas de dados e os algoritmos utilizados. A Seção 3 fornece as instruções para a compilação e execução do programa. A Seção 4 apresenta a análise formal da complexidade de tempo e espaço. A Seção 5 detalha as estratégias de programação defensiva adotadas para se manter a robustez do código. A

Seção 6 expõe a análise experimental realizada. Por fim, a Seção 7 apresenta as conclusões do trabalho e a Seção 8, a bibliografia consultada.

2. Implementação

Para a implementação da solução, o código foi organizado de forma modular, seguindo a estrutura de diretórios exigida. A pasta *include* armazena os arquivos de cabeçalho (.hpp), que definem as interfaces dos Tipos Abstratos de Dados (TADs) e os protótipos das funções. A pasta *src* contém os arquivos de código-fonte (.cpp), onde as funcionalidades são de fato implementadas. A compilação, gerenciada pelo *Makefile*, gera os arquivos objetos no diretório *obj* e o executável final, *tp2.out*, no diretório *bin*.

As principais estruturas de dados utilizadas foram classes projetadas especificamente para o problema, evitando o uso da biblioteca *STL*. A classe *Vetor* implementa um vetor dinâmico genérico capaz de armazenar ponteiros *void**, redimensionando sua capacidade automaticamente conforme a necessidade, garantindo o armazenamento de demandas, trechos e paradas. O *Escalonador* foi implementado como um *Min-Heap* para gerenciar a fila de prioridade da Simulação de Eventos Discretos, ordenando os eventos cronologicamente para garantir que o próximo evento a ser processado seja sempre o de menor tempo. Os TADs de domínio incluem a *Demanda*, que armazena o estado da solicitação e coordenadas; a *Corrida*, responsável por agregar passageiros e calcular a eficiência da rota combinada; e as classes *Trecho* e *Parada*, que modelam a logística do transporte.

O programa principal opera em duas fases: o agrupamento das demandas e a simulação dos eventos. Na primeira fase, a função *agrupaCorridas* percorre as solicitações e utiliza uma abordagem gulosa para tentar combinar demandas pendentes, verificando restrições de capacidade do veículo, intervalo temporal e proximidade geográfica, além de validar se a eficiência da rota combinada supera o limiar. Após a formação das corridas, a função *executaSimulacao* consome os eventos do escalonador, avançando o relógio da simulação instante a instante e processando a chegada dos veículos aos finais de trecho, gerando as estatísticas finais ao concluir cada trajeto.

Os testes do programa foram realizados em um ambiente com o sistema operacional Ubuntu 24.04.3, utilizando a linguagem C++. A compilação do código foi feita com o compilador GCC em sua versão 13.3.0. O hardware utilizado para a

execução dos testes foi um processador Intel Core i5-12400F e 32GB de memória RAM DDR4.

3. Instruções de compilação e execução

Para compilar e executar o programa, siga os passos descritos abaixo:

- Acesse o diretório raiz do projeto através do seu terminal;
- Execute o *Makefile* para compilar o projeto utilizando o seguinte comando: “*make all*”;
- Com esse comando, os arquivos de código-fonte serão compilados, os arquivos objeto (.o) serão gerados na pasta *obj* e o arquivo executável, chamado *tp2.out*, será criado na pasta *bin*;
- Para executar o programa, utilize o seguinte comando, que redireciona um arquivo de entrada para o programa: “*./bin/tp2.out < caminho/para/seu/arquivo_de_entrada.txt*”;
- A saída do programa, contendo as cenas geradas, é mostrada diretamente no terminal. Caso queira salvar a saída em um arquivo, proceda da seguinte maneira: “*./bin/tp2.out < caminho/para/seu/arquivo_de_entrada.txt > arquivo_de_saida.txt*”.

4. Análise de complexidade

Função\Complexidade	Tempo	Espaço
<i>Escalonador::insereEvento</i>	A estrutura utilizada é um <i>MinHeap</i> . A inserção adiciona o elemento no fim do vetor e realiza a operação de <i>heapify up</i> , percorrendo a altura da árvore. Sendo E o número de eventos, a complexidade é $O(\log E)$.	A função utiliza apenas o espaço para armazenar o novo nó no vetor dinâmico e variáveis auxiliares. Portanto a complexidade de espaço é $\theta(1)$.
<i>Escalonador::retiraProximoEvento</i>	Esta operação remove a raiz do <i>heap</i> , coloca o último elemento na raiz e realiza o <i>heapify down</i> para reordenar a árvore. Assim como a inserção, depende da altura da árvore, resultando em $O(\log E)$.	Não há alocação de memória dinâmica adicional para a operação em si, apenas manipulação de ponteiros e índices. Logo, a complexidade é $\theta(1)$.
<i>Corrida::construirRota</i>	Esta função itera sobre o número de passageiros da	A função aloca memória para vetores de paradas e

	corrida para criar paradas e trechos. Sendo η (<i>eta</i>) a capacidade máxima do veículo, o loop executa η vezes. Logo, a complexidade é $O(\eta)$, ou $O(1)$ assintoticamente em relação a N .	trechos proporcionais ao número de passageiros. O espaço é linear em relação à capacidade do veículo. Portanto, a complexidade é $O(\eta)$.
<i>agrupaCorridas</i>	Esta função contém dois laços aninhados. O laço externo percorre todas as N demandas. O laço interno busca candidatos para compartilhamento entre as demandas subsequentes. No pior caso a complexidade tende a $O(N^2)$.	A função cria novas instâncias de <i>Corrida</i> e <i>Evento</i> para serem armazenadas nos vetores principais. O espaço utilizado cresce linearmente com o número de corridas geradas, ou seja, $O(N)$.
<i>executaSimulacao</i>	A simulação processa eventos retirados do escalonador. Cada demanda gera um número constante de eventos. Portanto, E é proporcional a N . Como cada operação no heap custa $O(\log E)$, o custo total para processar todos os eventos é $O(N \log N)$.	A função opera retirando e deletando eventos da memória. O espaço auxiliar utilizado para processar cada evento individualmente é constante, ou seja, $\theta(1)$.

Em relação ao programa como um todo, o desempenho é dominado pela etapa de agrupamento de corridas na função *agrupaCorridas*. Devido à natureza combinatória da verificação de compartilhamento de corridas, onde cada demanda é comparada com demandas futuras, o algoritmo apresenta um comportamento quadrático no pior caso. Portanto, a complexidade de tempo total do sistema é $O(N^2)$.

Para a complexidade de espaço, temos que o custo será $O(N)$. O sistema armazena todas as demandas, cria objetos de corrida para atendê-las e popula o escalonador com eventos. Como as estruturas de dados principais (*Vetor* e *Escalaonador*) crescem linearmente conforme a entrada de dados aumenta, o consumo de memória é linear em relação à entrada.

5. Estratégias de Robustez

Para garantir a estabilidade e o funcionamento correto do sistema de simulação, diversas estratégias de programação defensiva e tolerância a falhas foram implementadas, com foco na prevenção de erros de tempo de execução e no tratamento de entradas inválidas. A gestão de memória foi uma prioridade, sendo tratada através da verificação rigorosa das alocações dinâmicas. Nas classes *Vetor* e *Escalonador*, a utilização do operador `new (std::nothrow)` assegura que o programa não seja encerrado abruptamente em caso de falha na alocação; em vez disso, o ponteiro resultante é verificado e, se nulo, uma mensagem de erro descritiva é emitida antes de uma saída controlada, prevenindo acessos inválidos à memória.

A robustez na manipulação de dados de entrada e na lógica de domínio foi reforçada através do tratamento de exceções e validação de estados. O programa principal foi encapsulado num bloco *try-catch* capaz de capturar erros de execução lançados pelas classes de domínio, como a tentativa de criar um *Trecho* com velocidade nula ou paradas inexistentes, garantindo que o erro seja reportado ao utilizador na saída de erro padrão (*stderr*) antes do encerramento. Adicionalmente, a leitura dos parâmetros de simulação e das demandas inclui verificações do estado do fluxo de entrada (*std::cin*), interrompendo a execução caso o formato do arquivo não corresponda ao esperado, o que impede a propagação de dados corrompidos para as estruturas de dados internas.

Por fim, a integridade lógica da simulação é mantida através do encapsulamento e da validação de pré-condições nos Tipos Abstratos de Dados. A classe *Demanda* gerencia seus estados internos (*DEMANDADA*, *INDIVIDUAL*, *COMBINADA*), impedindo que operações inválidas sejam realizadas sobre solicitações já processadas. O algoritmo de agrupamento verifica explicitamente a validade dos ponteiros e a consistência das distâncias antes de confirmar qualquer combinação de corridas, assegurando que as restrições de negócio, como a capacidade do veículo e o intervalo máximo de tempo, sejam estritamente respeitadas durante toda a execução do programa.

6. Análise Experimental

Esta seção apresenta a análise de desempenho do simulador de despacho, com foco em entender o impacto dos diferentes parâmetros de entrada na formação de corridas compartilhadas. O objetivo é avaliar as dimensões propostas na especificação do trabalho: “Candidatos a combinação” e “Custo de coleta e de entrega”.

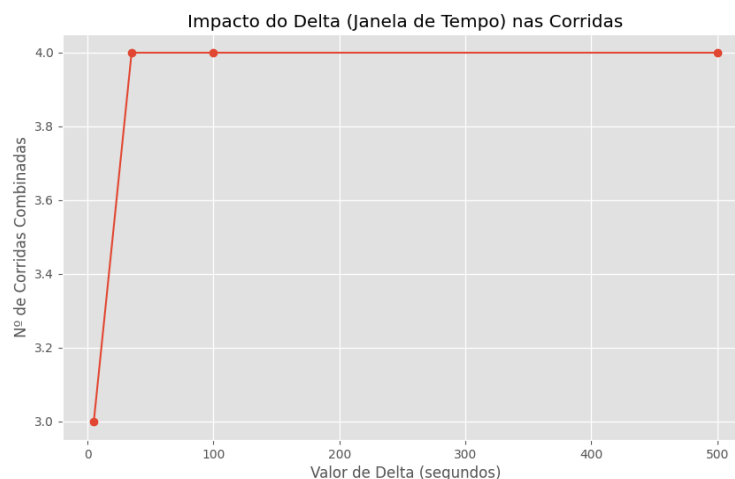
Para todos os experimentos, foi utilizado um conjunto de dados base fixo, composto pelas 48 demandas do arquivo *input_3.txt*. Uma “configuração base” de parâmetros ($\eta=3$, $\delta=35.0$, $\lambda=0.1$) foi definida. A partir dela, cada experimento variou um único parâmetro, mantendo os demais fixos, para isolar seu impacto. As métricas de avaliação principais foram o **Número de Corridas Combinadas** e o **Número Total de Passageiros Combinados**, dados extraídos diretamente da saída da simulação

Experimento 1 - Variação no *delta* (Intervalo Temporal)

Objetivo: avaliar a dimensão “Candidatos a combinação”

Hipótese: aumentar o delta (janela de tempo para agrupar corridas) deve aumentar o número de corridas combinadas, pois mais demandas serão consideradas candidatas.

Resultados: os testes variaram o delta entre 5.0 e 500.0



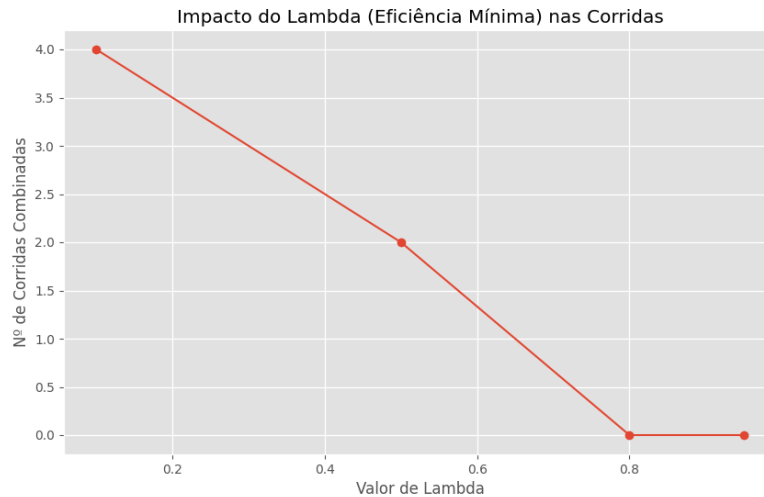
Análise: a hipótese foi confirmada apenas parcialmente. Reduzir a janela de tempo de fato prejudicou a formação de corridas, como esperado. No entanto, aumentar a janela para 100.0 ou 500.0 não trouxe benefício algum. Isso sugere que o valor de base já era suficiente para capturar todas as combinações viáveis. As demandas que ficaram fora dessa janela (mas dentro das janelas maiores) provavelmente foram reprovadas por outros critérios mais restritivos, como o λ ou os parâmetros de distância. O sistema atingiu um “ponto de saturação”, onde aumentar o *delta* não gera mais combinações.

Experimento 2 - Variação do λ (Eficiência Mínima)

Objetivo: avaliar a dimensão “Custo de coleta e de entrega”.

Hipótese: aumentar o λ (eficiência mínima exigida) deve diminuir drasticamente o número de corridas combinadas, pois o critério de qualidade se torna mais rigoroso.

Resultados: os testes variaram o λ de 0.1 a 0.95



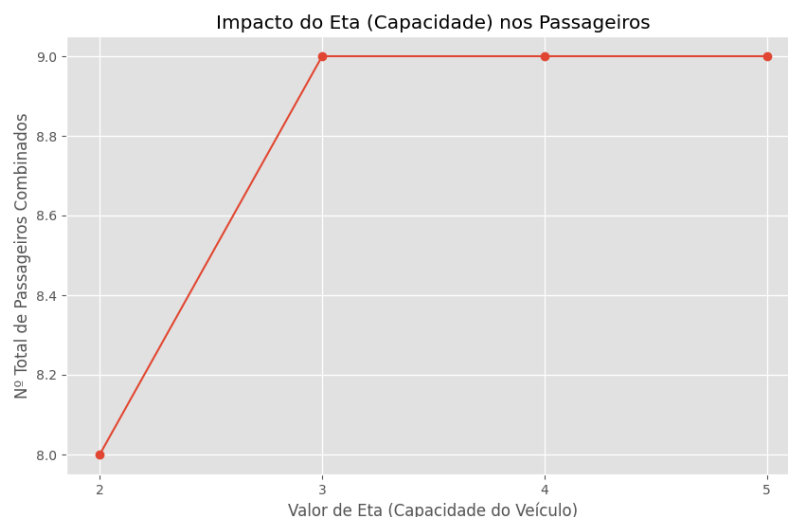
Análise: a hipótese foi fortemente confirmada. Os dados demonstram que o λ é o parâmetro de maior sensibilidade e o principal “gargalo” na formação de corridas. Um aumento de 0.1 para 0.5 cortou o número de combinações pela metade. Ao exigir uma eficiência de 80% ($\lambda=0.8$), o sistema considerou que nenhuma das combinações possíveis valia a pena, optando por executar apenas corridas individuais. Isso indica que as combinações possíveis neste conjunto de dados possuíam um “custo” de coleta/entrega que as tornava, em média, menos eficientes que 80%.

Experimento 3 - Variação do η (Capacidade do Veículo)

Objetivo: avaliar a dimensão “Candidatos a combinação”.

Hipótese: aumentar o η (capacidade máxima de passageiros) deve permitir mais passageiros por corrida, aumentando o total de passageiros combinados.

Resultados: os testes variaram o η de 2 a 5 passageiros.



Análise: a hipótese foi parcialmente confirmada. O valor $\eta=2$ foi claramente restritivo; ao analisar os dados, nota-se que uma corrida que poderia ter 3 passageiros foi limitada a 2, resultando em 8 passageiros totais em vez de 9. Contudo, aumentar a capacidade para 4 ou 5 não trouxe benefício. O número de passageiros combinados estagnou em 9. Isso demonstra a “lei dos rendimentos decrescentes” para este parâmetro: não adianta aumentar a capacidade do veículo se nenhuma combinação viável com 4 ou 5 passageiros foi encontrada no conjunto de dados. O gargalo, novamente, não era a capacidade, mas sim os outros critérios de combinação.

Os experimentos demonstram de forma consistente que os parâmetros de simulação têm impactos muito distintos e que o sistema é sensível a “gargalos”.

O parâmetro λ é o filtro mais impactante. Pequenos aumentos no limiar de eficiência reduzem drasticamente o número de corridas compartilhadas, chegando a eliminá-las.

Os parâmetros δ e η apresentaram um “ponto de suficiência”. Aumentar seus valores além de um certo ponto ($\delta > 35$ e $\eta > 3$) não gerou melhorias, pois outros parâmetros atuaram como o gargalo limitante.

Para o conjunto de dados analisado, o sistema de combinação é robusto, mas altamente dependente da eficiência das rotas geradas, sendo menos sensível à janela de tempo ou à capacidade máxima dos veículos, desde que estes não sejam excessivamente restritivos.

7. Conclusão

Este trabalho lidou com o problema de despacho e roteamento de veículos em sistemas de transporte por aplicativo, com foco na otimização de corridas compartilhadas. A abordagem utilizada para sua resolução foi a implementação de um Simulador de Eventos Discretos em C++, estruturado sobre uma arquitetura modular que separa a lógica de agrupamento de demandas da execução temporal da simulação. Com a solução adotada, pode-se verificar que a aplicação de uma heurística gulosa, fundamentada em restrições rígidas de capacidade, tempo e espaço, permite a formação eficiente de rotas compartilhadas, reduzindo o número total de viagens necessárias sem violar os limites de conforto e eficiência estipulados pelos parâmetros do sistema.

Por meio da resolução desse trabalho, foi possível praticar os conceitos relacionados à implementação de estruturas de dados fundamentais sem o auxílio de bibliotecas padrão, especificamente a construção de vetores dinâmicos e filas de prioridade baseadas em *Min-Heap*. O desenvolvimento reforçou o entendimento sobre o gerenciamento manual de memória em C++, a importância do encapsulamento através de Tipos Abstratos de Dados e a lógica de simulação baseada no avanço de tempo por eventos, uma técnica essencial para a modelagem de sistemas complexos e estocásticos.

Durante a implementação da solução para o problema, houveram importantes desafios a serem superados. A gestão correta dos ponteiros e a liberação de memória exigiram atenção constante para evitar vazamentos e falhas de segmentação. Outro desafio significativo foi a implementação robusta do algoritmo de agrupamento, que demandou a validação simultânea de múltiplas restrições geométricas e temporais, bem como o tratamento de casos de borda na leitura de arquivos de entrada para garantir que o simulador não entrasse em estados inconsistentes diante de dados malformados.

8. Bibliografia

Cunha, W., Lacerda, A., Meira Jr., W., Santos, M. (2025). Slides virtuais da disciplina de estrutura de dados.

Disponibilizados via moodle. Departamento de Ciência da Computação.

Universidade Federal de Minas Gerais. Belo Horizonte.